SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 — Concept Mapping (Annexure A)

Field	Details	Field	Details
Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO1	Assessment:	Assessment Tool 2 – Concept Mapping
Weightage:	35 %	Total Marks:	25
Student Name:	Tharun G	Reg. No.:	192511144
Department:	CSE	Date:	28-07-2026

CO1 Statement

Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking. (BL4)

Code of Conduct

I, Tharun G / 192511144 (Name / Reg. No.), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____Tharun_____ Date: 25-07-2026

Annexure A — Index of Questions

Q	Concept Map Topic
1	CPU – Memory – I/O integration via single / multi / hierarchical bus structures
2	Instruction execution cycle (Fetch–Decode–Execute) linked to CPI, clock cycles and benchmarking
3	8085 vs 8086 architecture: instruction set, registers, addressing modes, segmentation
4	Addressing modes linked to instruction types and their impact on speed / size / memory access
5	Number systems (binary, hexadecimal) in instruction representation, storage and debugging

Question 1 — CPU, Memory & I/O via Bus Structures

Problem: *Construct a concept map showing CPU (ALU, Control Unit, Registers), memory (cache, RAM) and I/O devices, extended to single-, multi- and hierarchical-bus structures, showing data/control flow, bottlenecks and how bus structure improves performance.*

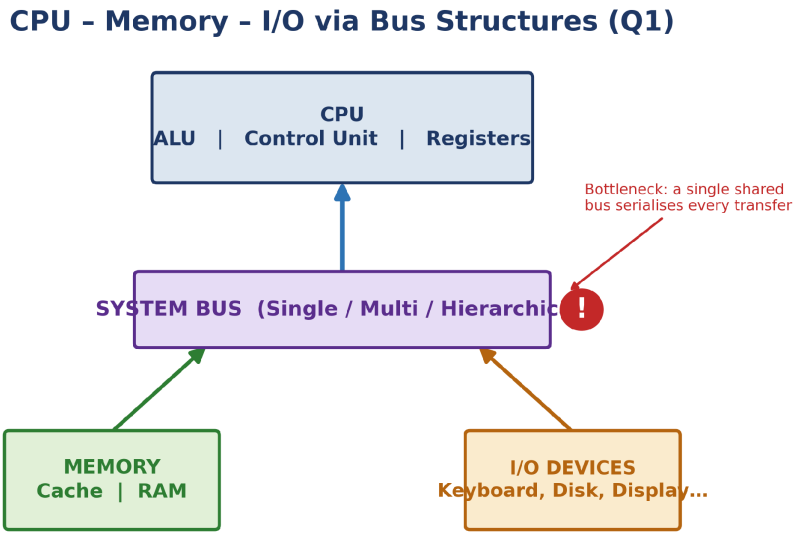

Figure 1.1 — CPU / Memory / I/O linked through the system bus, with the bottleneck point marked

Concept Identification: CPU sub-units (ALU, Control Unit, Registers), memory sub-units (Cache, RAM), I/O devices, and the three bus variants — single, multi-, and hierarchical-bus — are all identified as the map's core nodes.

Linkages: The bus is the single link connecting all three subsystems: the Control Unit issues control signals over the bus to time every transfer, while address and data lines carry the actual read/write traffic between CPU, memory and I/O.

Organization: The map is hierarchical with the bus as the central node and CPU, Memory and I/O as three peer branches — correctly showing the bus as shared infrastructure rather than a property of any one subsystem.

Integration & Application: A single shared bus (cheap, used in early microprocessors) serializes every transfer and is the main bottleneck; multi-bus and hierarchical-bus designs (e.g. a dedicated CPU–memory bus plus a slower peripheral bus, as in modern PCIe systems) relieve this by giving fast and slow devices separate paths.

Clarity: Because only one device can use a shared bus at a time, contention there is the chief source of delay; separating high-speed CPU–memory traffic from slower I/O traffic in a multi-bus or hierarchical layout is what improves overall throughput.

Question 2 — Instruction Cycle & CPU Performance

Problem: *Develop a concept map linking the instruction execution cycle (fetch, decode, execute) with CPI, clock cycles and execution time, including memory access, instruction complexity, pipeline stalls, and benchmarking.*

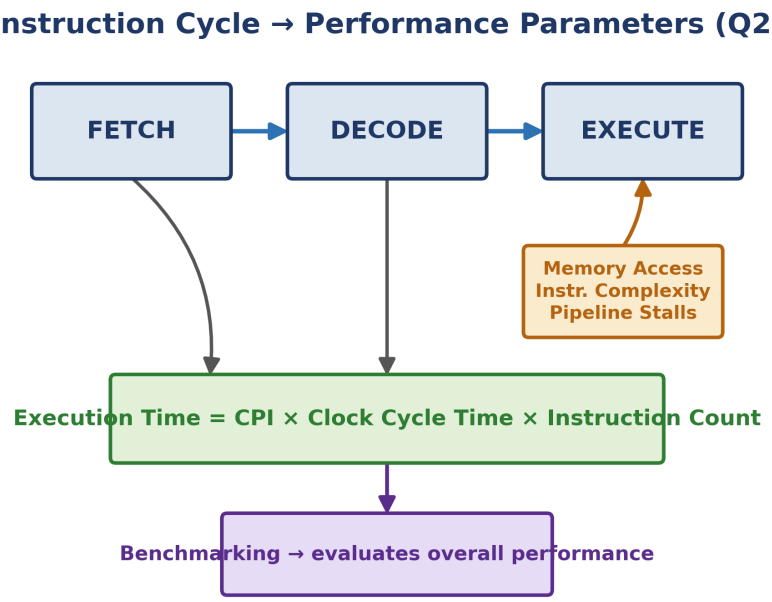

Figure 2.1 — Fetch–Decode–Execute chain linked to the execution-time formula and benchmarking

Concept Identification: The three cycle stages (Fetch, Decode, Execute), the performance parameters CPI and clock cycle time, and the influencing factors — memory access, instruction complexity, pipeline stalls — plus benchmarking are all named on the map.

Linkages: Each stage consumes clock cycles; CPI is the average of these across an instruction mix, and Execution Time = CPI × Clock Cycle Time × Instruction Count is the formula linking the cycle directly to a measurable performance number.

Organization: A linear three-stage chain (Fetch→Decode→Execute) feeds one central formula box, with the three influencing factors shown branching specifically into the Execute stage, where they actually take effect.

Integration & Application: A pipeline stall — e.g. a cache miss during Execute — inflates the effective CPI above its ideal value; benchmarking (SPEC-style suites) measures this real, stall-inclusive performance rather than a theoretical best case, which is why it is placed as the map's final evaluation node.

Clarity: Performance is fully explained by how many cycles the three stages need (CPI), how fast the clock runs, and how many instructions run — with stalls and memory access raising CPI in practice, which benchmarking exists to reveal.

Question 3 — 8085 vs 8086 Architecture Comparison

Problem: *Create a concept map comparing 8085 and 8086 architectures — instruction sets, register organization, addressing modes, and memory segmentation (8086) — and show how these influence execution, multitasking and real-time efficiency.*

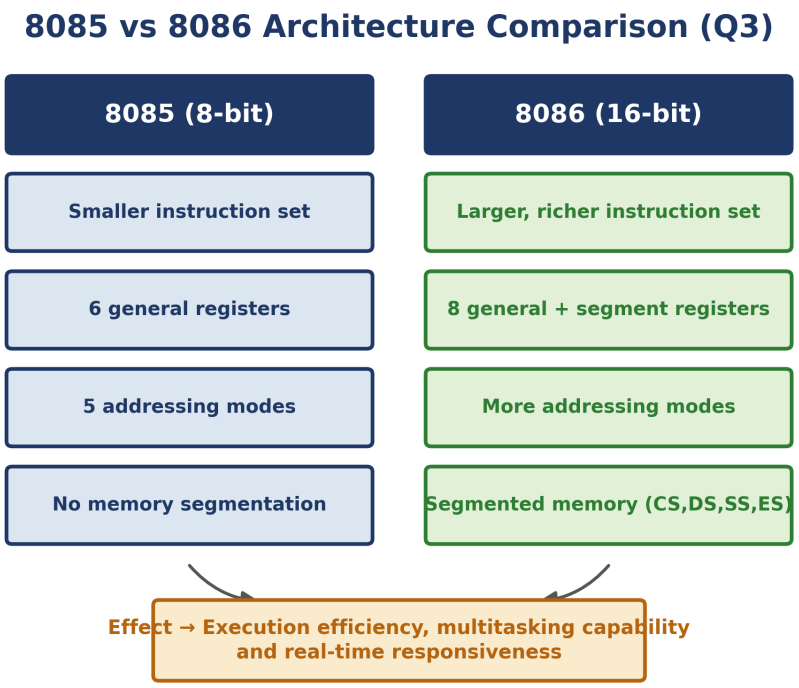

Figure 3.1 — Row-aligned comparison of 8085 and 8086, converging on execution efficiency and multitasking

Concept Identification: The four compared features — instruction-set size, register organization, addressing modes and memory segmentation — are identified for both the 8-bit 8085 and the 16-bit 8086.

Linkages: 8086 extends 8085 in every row: wider registers permit a larger instruction set, and the added segment registers (CS, DS, SS, ES) are what make segmented memory addressing possible — a capability 8085 simply has no register to support.

Organization: The map is a direct two-column, row-aligned comparison, so each 8085 trait sits opposite its corresponding 8086 trait, making the architectural upgrade immediately visible feature-by-feature.

Integration & Application: Segmentation lets 8086 address a full 1 MB versus the 8085's 64 KB, and separate code/data/stack segments give it a basic multitasking capability — relevant to real-time applications that need larger, more structured memory than 8085 can offer.

Clarity: 8086's wider registers, richer instruction set and segmented memory together give it greater execution efficiency and multitasking capacity than 8085, at the cost of increased architectural complexity.

Question 4 — Addressing Modes, Instruction Types & Impact

Problem: *Draw a concept map connecting addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control), and show their effect on memory-access time, instruction size and execution speed.*

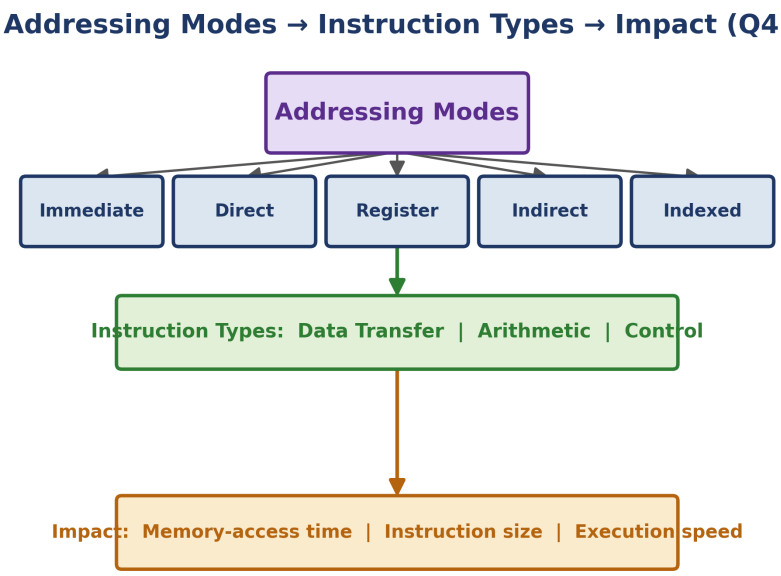

Figure 4.1 — Five addressing modes converging on instruction types and their combined performance impact

Concept Identification: All five addressing modes (Immediate, Direct, Register, Indirect, Indexed), the three instruction-type categories (Data Transfer, Arithmetic, Control), and the three impact factors (memory-access time, instruction size, execution speed) are identified.

Linkages: Each mode is chosen according to the instruction type it serves — Immediate suits constant-loading Data Transfer instructions, while Indexed suits array-traversal Arithmetic instructions — and every mode ultimately determines the same three impact factors.

Organization: The map is a tree: Addressing Modes at the root branches into the five modes, which converge into a shared Instruction Types band and then a shared Impact band, correctly representing the many-to-one relationship between modes and their effects.

Integration & Application: Register addressing needs no memory access at all, so compilers prefer it inside tight loops; Indirect and Indexed modes cost extra memory access and produce larger instructions but are indispensable for pointers and array traversal in real programs.

Clarity: Choice of addressing mode trades instruction size and memory-access time against flexibility: Register mode is fastest and smallest, while Indirect/Indexed are slower but necessary wherever data is accessed dynamically.

Question 5 — Number Systems in Instruction Handling

Problem: *Prepare a concept map integrating binary and hexadecimal number systems with instruction representation, memory storage and debugging, showing format conversion and why hex is preferred in low-level programming.*

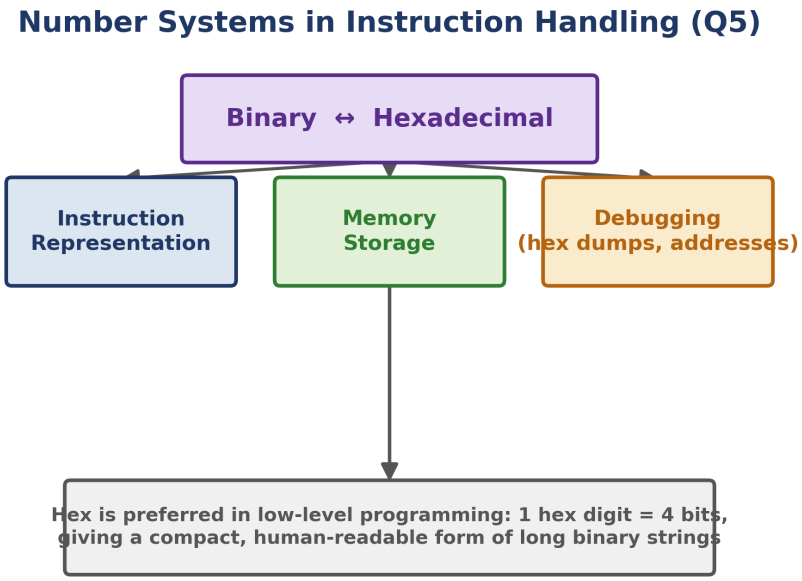

Figure 5.1 — Binary–Hexadecimal conversion feeding instruction representation, memory storage and debugging

Concept Identification: Binary and Hexadecimal number systems are identified as the root concept, linked to three application areas: instruction representation, memory storage, and debugging.

Linkages: Every instruction and address is ultimately stored as binary; hexadecimal is a direct, lossless re-encoding of that same binary (one hex digit = exactly four bits), which is why all three application leaves trace back to the same binary–hex relationship.

Organization: A root node (Binary ↔ Hexadecimal) branches into three parallel leaves — representation, storage, debugging — each depending on the identical underlying conversion, shown converging back into one explanatory note at the base of the map.

Integration & Application: Debuggers and memory dumps display addresses and opcodes in hex because a 32-bit address needs only 8 hex digits versus 32 binary digits — cutting transcription errors during low-level debugging while remaining an exact view of the underlying bits.

Clarity: Binary is what the CPU actually stores and executes; hexadecimal is used throughout instruction representation, memory displays and debugging purely as a compact, human-readable shorthand for that same binary data.

Marks Allocation

Criteria →	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

Each question's concept map has been organised under exactly these five criteria — Concept Identification, Linkages, Organization, Integration & Application, and Clarity — so that marks can be awarded section-by-section directly against the rubric on the following page.

Rubrics for Calculation

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Signature of the Student **Signature of the Faculty**